VIT — Vellore Institute of Technology

**Internet of Things**

**Smart Security & Light Control System**

Project Report

| | |
|---|---|
| **Name** | Sanat Vasisht |
| **Roll Number** | 24BEC0411 |
| **GitHub Repository** | VIT/AIIoT/24BEC0411_SANAT_VASISHT |
| **Simulation Tool** | Wokwi |
| **Cloud Platform** | ThingSpeak |
| **Programming Language** | Embedded C (Arduino Framework) |
| **IDE** | Wokwi Online Simulator |

# 1. Aim

To design and simulate a Smart Security and Light Control System that automatically detects human movement and ambient light levels, controls lighting, and sends real-time cloud alerts.

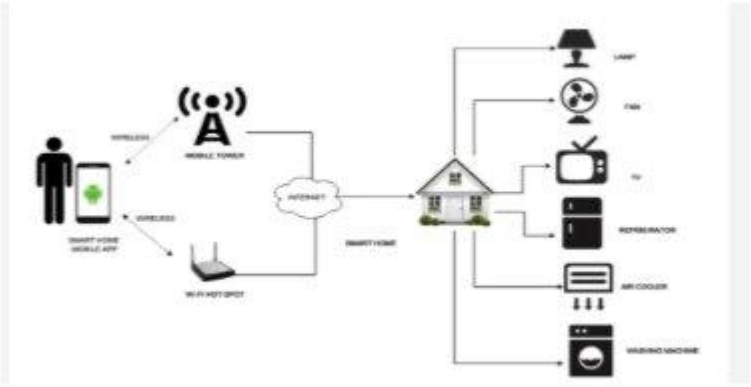


*Figure 1: Smart Home IoT System Overview — sensors and devices connected via internet to a central smart home controller*

# 2. Problem Statement

## 2.1 Define the Problem

I n modern residential and commercial environments, undetected intrusions in low-visibility conditions pose a safety risk, while unnecessarily powered lights contribute to energy wastage. There is a pressing need for a system that can intelligently detect human presence in dark environments and respond autonomously — activating local lighting and alerting remote parties through the cloud — without requiring manual intervention.


*Figure 2: Smart security threat scenario — motion detected in low-light conditions triggering real-time alert*

## 2.2 Context and Significance

The Internet of Things (IoT) has enabled a new generation of smart systems where physical sensors communicate with cloud platforms to enable real-time monitoring and automated responses. This project is significant because:

- It addresses both security and energy efficiency simultaneously through a single intelligent system
- It leverages low-cost IoT hardware (PIR + LDR + microcontroller) making it applicable in homes, warehouses, and industrial premises
- Cloud connectivity via ThingSpeak enables remote monitoring, logging, and alerting — aligning with Industry 4.0 standards
- The conditional logic (motion AND darkness) ensures lights activate only when necessary, reducing unnecessary power consumption

## 2.3 Existing Solutions and Their Limitations

Several approaches exist for home security and lighting automation, each with notable limitations:

- **Manual light switches:** Require user presence and are prone to human error with no remote monitoring capability
- **Timer-based systems:** Operate on fixed schedules, do not adapt to actual presence or ambient conditions, causing energy waste
- **Standalone CCTV cameras:** Passive recording only, do not trigger automated lighting or real-time alerts without expensive infrastructure
- **Commercial smart home platforms (Google Home, Amazon Alexa):** High cost, vendor lock-in, limited customizability, not suitable for industrial or educational implementation

This project addresses these gaps by implementing a low-cost, fully simulated, cloud-connected smart security and lighting system combining motion detection, ambient light sensing, automated control, and real-time ThingSpeak cloud alerting in a single integrated solution.

# 3. Scope of the Solution

## 3.1 Boundaries and Objectives

This project is scoped to simulate a smart home security and lighting system with the following objectives:

- Detect human motion using a PIR sensor and measure ambient light using an LDR sensor
- Automatically control an LED output based on combined sensor conditions
- Send real-time data and alerts to ThingSpeak cloud when the trigger condition is met
- Demonstrate the complete system via Wokwi simulation with screen-recorded video and audio narration

## 3.2 Solution Aims

- Provide an automated, energy-efficient alternative to manual lighting and standalone security systems
- Simulate a production-ready IoT pipeline: Sense → Process → Actuate → Cloud
- Demonstrate industrial IoT principles using accessible tools (Wokwi + ThingSpeak + Embedded C)

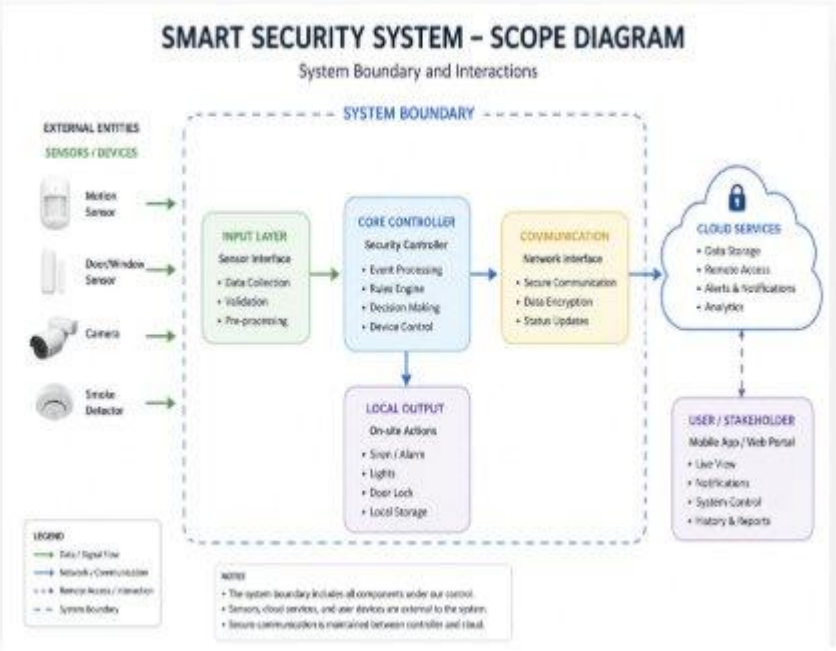


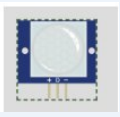

*Figure 3: Smart Security System — Scope Diagram showing system boundary, input layer (sensors), core controller, local output, communication interface, and cloud services*

## 3.3 Constraints and Limitations

- PIR sensors can trigger false alarms from pets, moving curtains, or heat sources
- Wi-Fi dependent — any network outage cuts off cloud alerts entirely
- No facial recognition — cannot distinguish between residents and intruders

# 4. Required Components to Develop Solution

## 4.1 Hardware Components (Simulated in Wokwi)

| Image | Component | Purpose |
|---|---|---|
|  | **ESP32 Microcontroller** | Brain of the system — processes sensor data, controls LED, sends Wi-Fi alerts to ThingSpeak |
|  | **PIR Sensor (HC-SR501)** | Detects infrared radiation from human body movement |
|  | **LDR (Photoresistor)** | Measures ambient light level via voltage divider circuit |

| Component | Model / Spec | Purpose |
|---|---|---|
| Microcontroller | ESP32 (Xtensa LX6, 240MHz) | Main processing unit with built-in Wi-Fi — reads sensors, controls LED, sends cloud alerts |
| Motion Sensor | PIR Sensor (HC-SR501) | Detects infrared radiation from human body movement |
| Light Sensor | LDR (Photoresistor) | Measures ambient light level via voltage divider circuit |
| Output Device | LED (Red, 5mm) | Simulates the automated security light |
| Resistor | 10kΩ (LDR pull-down) | Forms voltage divider with LDR for analog reading |
| Resistor | 220Ω (LED current limit) | Prevents excess current through LED |
| readboard | 150-point board | Component mounting and wiring |

## 4.2 Software Components

| Software / Tool | Purpose |
|---|---|
| Wokwi (wokwi.com) | Browser-based circuit simulation — place components, wire, and run code |
| Embedded C (Arduino Framework) | Programming language used for sensor logic and control |
| ThingSpeak (thingspeak.com) | Cloud IoT platform — receives, stores, and visualizes sensor data |
| GitHub | Version control and project submission repository |


*Figure 4: Wokwi simulator and Embedded C — tools used for simulation and programming*

## 4.3 Libraries and Frameworks

| Library | Purpose |
|---|---|
| Arduino.h | Core Arduino framework functions |
| WiFi.h (ESP32 variant) | Wi-Fi connectivity for ThingSpeak communication |
| HTTPClient.h | HTTP GET requests to ThingSpeak REST API |

## 4.4 Cloud Environment

| Platform | Details |
|---|---|
| ThingSpeak | https://thingspeak.mathworks.com/channels/3409155 |
| ThingSpeak Channel | Field 1: PIR Status (0/1), Field 2: LDR Value, Field 3: LED Status |
| ThingSpeak API | REST API — HTTP GET request with Write API Key from microcontroller |

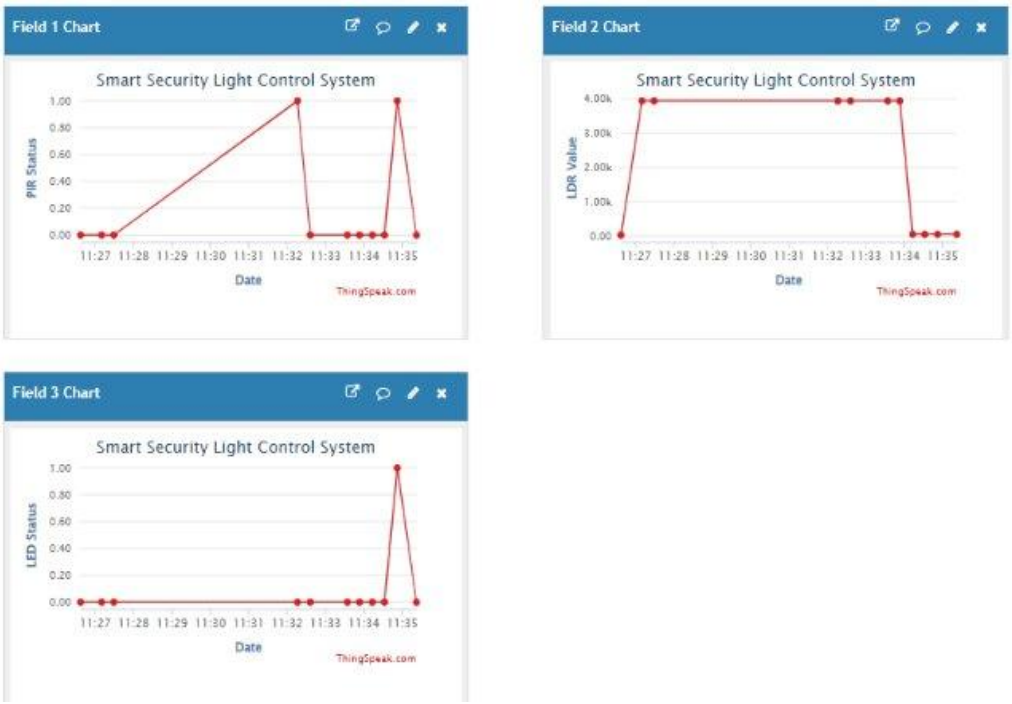



*Figure 5: ThingSpeak Real-Time Dashboard — Smart Security Light Control System*

11:27 — Dark, no motion → LED OFF | 11:28–11:31 — Bright, no motion → LED OFF
11:32 — Bright + motion → LED OFF (no alert) | 11:34 — Dark + motion → LED ON, alert sent ✓

# 5. Flowchart of the Code

The following flowchart describes the complete logic executed by the ESP32 microcontroller.

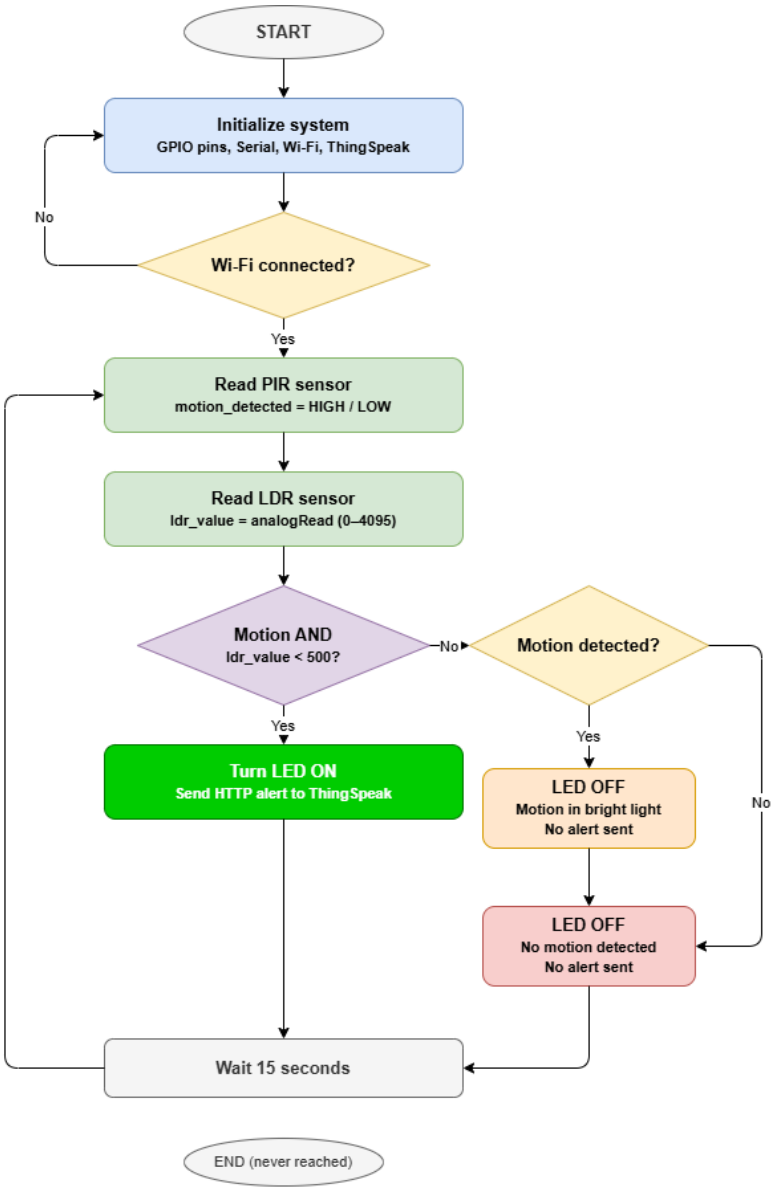



*Figure 6: System flowchart — complete ESP32 firmware logic from initialization to cloud alert*

## 5.1 Flowchart Logic Description

The firmware follows this sequence:

1. START — Initialize GPIO pins, Serial communication, Wi-Fi connection
2. Check Wi-Fi connection — retry until connected
3. Read PIR sensor — motion_detected = HIGH or LOW
4. Read LDR sensor — ldr_value = analogRead (0 to 4095)
5. Decision — IF motion_detected AND ldr_value < 50 (dark)
6. YES → Turn LED ON → Send HTTP alert to ThingSpeak
7. NO → Check if motion detected:

- Motion + bright light → LED OFF, no alert
- No motion → LED OFF, no alert

8. Wait 15 seconds → Repeat from Step 3

# 6. Simulated Circuit (Wokwi)

The circuit is built and simulated entirely on Wokwi (wokwi.com). The simulation includes all components wired as they would be in a real physical prototype.

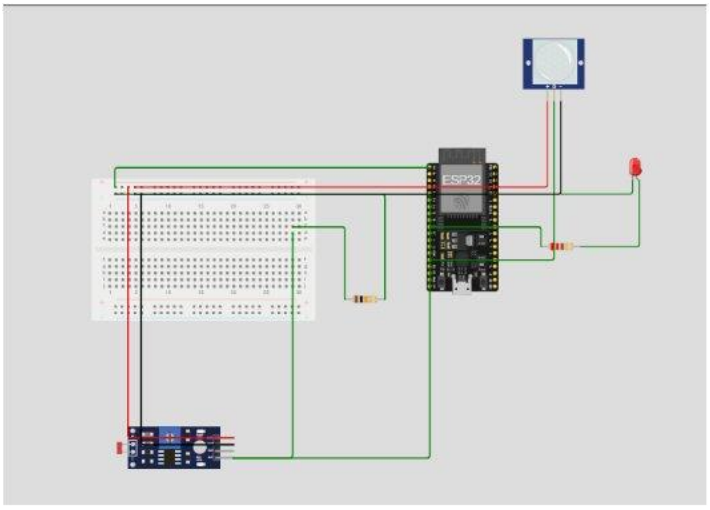



*Figure 7: Complete Wokwi simulation circuit — ESP32 connected to PIR sensor, LDR module, and LED*

## 6.1 Circuit Connections

| Component | Pin | ESP32 Pin |
|---|---|---|
| PIR Sensor | VCC | 3.3V |
| PIR Sensor | GND | GND |
| PIR Sensor | OUT | GPIO 13 |
| LDR + 10kΩ | Junction (divider) | GPIO 34 |
| LDR | One end | 3.3V |
| 10kΩ Resistor | Other end of LDR | GND |
| LED Anode (+) | Via 220Ω resistor | GPIO 26 |
| LED Cathode (−) | Direct | GND |

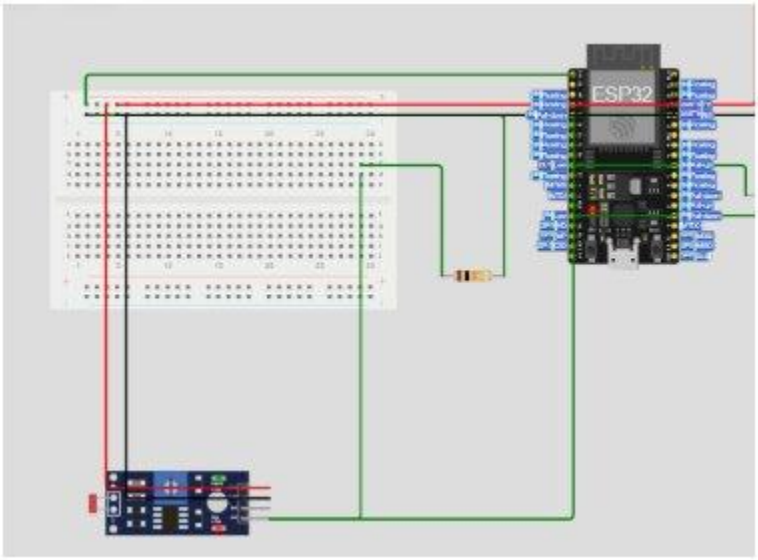



*Figure 8: Wokwi circuit — LDR module and 10kΩ voltage divider connected to ESP32 GPIO 34*

## 6.2 Circuit Functionality

- The PIR sensor outputs HIGH on GPIO 13 when it detects infrared radiation from a moving person
- The LDR and 10kΩ resistor form a voltage divider — in darkness, LDR resistance is high so voltage at GPIO 34 is low
- The ESP32 reads both values every loop iteration and applies the AND condition
- When both conditions are true, GPIO 26 goes HIGH turning the LED ON
- ESP32 sends real HTTP request to ThingSpeak via Wi-Fi

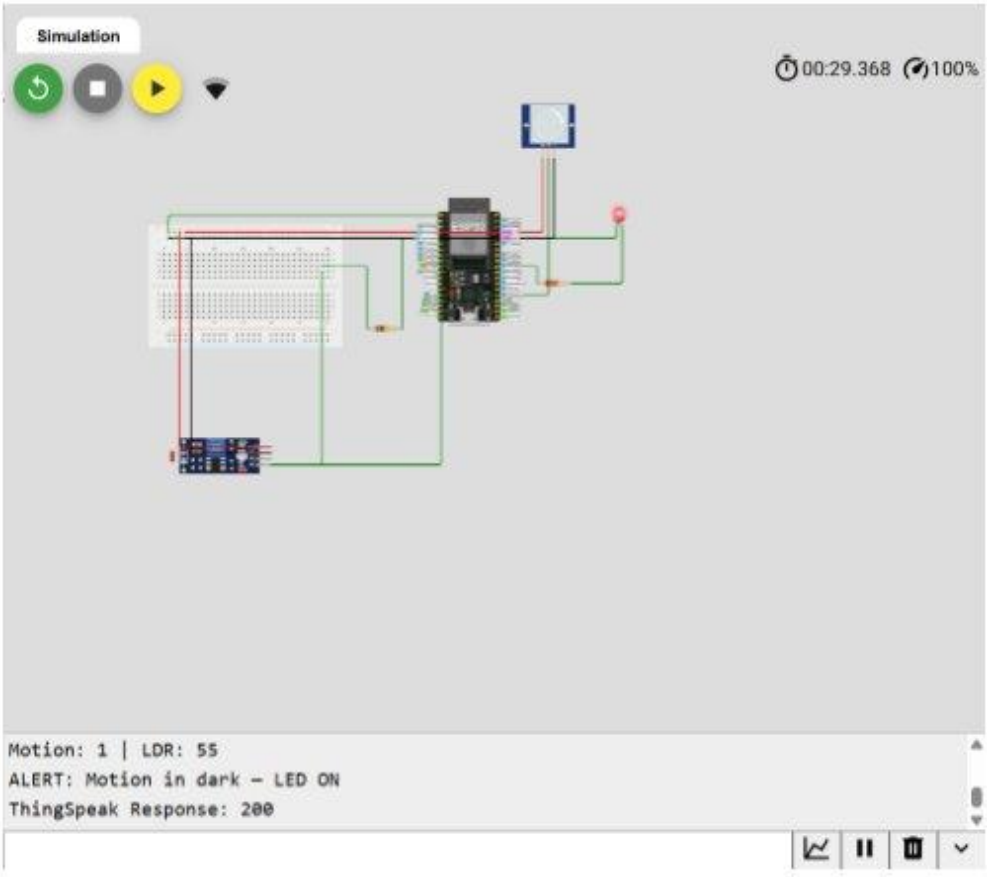



*Figure 9: Wokwi simulation — LED ON state with Serial Monitor confirming ALERT and ThingSpeak Response: 200*

# 7. Code for the Solution

**Language:** Embedded C (Arduino Framework)

**IDE:** Wokwi Online Simulator

**Cloud:** ThingSpeak (Channel ID: 3409155)

The code is organized into subsections: library includes, pin and parameter definitions, Wi-Fi initialization, ThingSpeak cloud function, and main loop logic.

## 7.1 Include Libraries

```
// Include required libraries for I/O and communication
#include <WiFi.h>         // For ESP32 Wi-Fi connectivity
#include <HTTPClient.h>   // For ThingSpeak HTTP requests
```

## 7.2 Define Parameters

```
// Define sensor pins
#define PIR_PIN       13     // PIR sensor output pin (digital)
#define LDR_PIN       34     // LDR analog input pin
#define LED_PIN       26     // LED output pin

// Define thresholds
#define LDR_THRESHOLD  500    // Below this = dark (0-4095 scale)

// Wi-Fi credentials
const char* ssid     = "Wokwi-GUEST";
const char* password = "";

// ThingSpeak configuration
const char* server   = "https://api.thingspeak.com/update";
const char* apiKey   = "QPKUOOPB0GTT26LQ";
const int channelID  = 3409155;
```

## 7.3 Initialization Functions

```
void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  // Connect to Wi-Fi
  Serial.print("Connecting to Wi-Fi");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Connected!");
  Serial.println("System Initialized");
}
```

## 7.4 Cloud Alert Function

```
// Send real-time data to ThingSpeak via HTTP GET
void sendToThingSpeak(int pirStatus, int ldrVal, int ledStatus) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String url = String(server) +
                 "?api_key=" + apiKey +
                 "&field1=" + pirStatus +
                 "&field2=" + ldrVal +
                 "&field3=" + ledStatus;
    http.begin(url);
    int httpCode = http.GET();
    Serial.print("ThingSpeak Response: ");
    Serial.println(httpCode);
    http.end();
  }
}
```

## 7.5 Main Program Loop

```
// Main loop — read sensors, control LED, send cloud alert
void loop() {
  int motionDetected = digitalRead(PIR_PIN);
  int ldrValue       = analogRead(LDR_PIN);

  Serial.print("Motion: "); Serial.print(motionDetected);
  Serial.print(" | LDR: "); Serial.println(ldrValue);

  int ledStatus = 0;

  if (motionDetected == HIGH && ldrValue < LDR_THRESHOLD) {
    digitalWrite(LED_PIN, HIGH);
    ledStatus = 1;
    Serial.println("ALERT: Motion in dark — LED ON");
  } else if (motionDetected == HIGH && ldrValue >= LDR_THRESHOLD) {
    digitalWrite(LED_PIN, LOW);
    ledStatus = 0;
    Serial.println("Motion in bright light — No alert");
  } else {
    digitalWrite(LED_PIN, LOW);
    ledStatus = 0;
    Serial.println("No motion detected — LED OFF");
  }

  // Send data to ThingSpeak every 15 seconds
  sendToThingSpeak(motionDetected, ldrValue, ledStatus);
  delay(15000);
}
```


*Figure 10: Wokwi code editor — complete Embedded C code for Smart Security Light Control System*

# 8. Video of the Demo

A screen recording of the Wokwi simulation demonstrating the complete system with audio narration. The video highlights key features and interactions of the Smart Security and Light Control System.

## 8.1 Video Link

**GitHub Repository:**
https://github.com/sanatv18/VIT_AIIoT_24BEC0411_SANAT_VASISHT

**Video demo link:**

https://drive.google.com/file/d/1JhmUyuPVtr0ZDdYzOFvZ4OGliZnjpOLZ/view?usp=drive_link

## 8.2 Demo Scenarios

The following scenarios are demonstrated in the video:

| # | Scenario | Action in Wokwi | Expected Output |
|---|---|---|---|
| 1 | Motion + Dark | PIR = HIGH, LDR covered (low lux) | LED ON, Serial: ALERT sent to ThingSpeak |
| 2 | Motion + Bright | PIR = HIGH, LDR exposed to light | LED OFF, no alert |
| 3 | No Motion + Dark | PIR = LOW, LDR covered | LED OFF, no alert |
| 4 | No Motion + Bright | PIR = LOW, LDR in light | LED OFF, no alert |

# 9. GitHub Repository

All project files are uploaded to the following GitHub repository as per L&T EduTech submission guidelines:

| Item | Details |
|---|---|
| Repository Name | VIT_AIIoT_24BEC0411_SANAT_VASISHT |
| Repository URL | https://github.com/sanatv18/VIT_AIIoT_24BEC0411_SANAT_VASISHT |
| Report (PDF) | /report/Smart_Security_Light_Control.pdf |
| Flowchart | smart_security_flowchart_v2.drawio + .png |
| Source Code | /code/smart_security.ino |
| Demo Video | https://drive.google.com/file/d/1JhmUyuPVtr0ZDdYzOFvZ4OGliZnjpOLZ/view?usp=drive_link |
| Wokwi Link | https://wokwi.com/projects/466968022326038529 |